

P2779R0

Make `basic_string_view`'s range construction conditionally explicit

Published Proposal, 2023-01-30

Author:

[Giuseppe D'Angelo](#)

Audience:

SG18, LEWG, SG16, SG9

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to relax the range constructor of `basic_string_view` so that that particular construction is implicit when used together with string types.

Table of Contents

- 1 Changelog**
- 2 Motivation and Scope**
- 3 Design Decisions**
 - 3.1 How to identify a string(view) class?
 - 3.2 Is this a defect fix?
- 4 Impact on the Standard**
- 5 Technical Specifications**

6 Proposed wording

- 6.1 Option 1: introduce a dedicated type trait
- 6.2 Option 2: use a compatible inner `type_traits` type

Appendix A: `traits_type` in Boost 1.81

7 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and Scope

[P1989R2] added a new constructor to `basic_string_view` that made it implicitly constructible from any *compatible* range (contiguous, sized, that uses the same character type as the string view, and so on; cf. [string.view.cons]/11).

[P2499R0] subsequently modified this constructor, by making it unconditionally `explicit`.

The reasoning for this change was that not all compatible ranges are convertible to a string view by using their own size (the "range size"). In many important use cases, the size that is meant to be used is the "`strlen` size" (intended as the distance of a given marker, usually `NUL`, from the beginning of the range). In those cases, having an implicit conversion towards string views may end up introducing serious bugs in a program.

For instance, [P2499R0] has this example featuring "Modern C++" datatypes:

```
extern void get_string(std::span<char> buffer);
extern void use_string(std::string_view str);
```

```
std::array<char, 200> buf;
get_string(buf);
use_string(buf); // oops
```

This code used to compile before [P2499R0]. However: if the semantics of `get_string` is to NUL-terminate its output, `use_string` will instead create a view over the entire array, spanning over the embedded NUL. This opens the door to any sort of bug (incl. UB, by reading uninitialized data in the array), and could have security implications. To prevent this from happening, the conversion from `buf` to `std::string_view` has been made explicit, and the example code above is now ill-formed.



This change has a few consequences.

First and foremost it's worth underlying that the proposed solution is still somehow a band-aid for this sort of use cases. If one uses `std::string s(200);` as a storage type instead of some other contiguous container (like `std::array<char, 200>`), then the example above would still have the problem of using the "wrong" size when `s` is converted to a `std::string_view`. The conversion from `std::string` to `std::string_view` remains however implicit.

Second, and more in general; in a sense, the proposed change splits the types that can be converted to a string view in two families:

1. types for which we now axiomatically establish that their size is always the "range size", and these convert implicitly (e.g. `std::string`);
2. types for which don't necessarily know which one is the "correct" size, and those must be converted explicitly after [P2499R0] (e.g. `std::vector<char>`).

The problem with split (and in particular with 2.) is that it fundamentally kills the usability of string views as an universal interface type for read-only non-owning string-like inputs.

For instance, if one has a string type different than `std::string`, then using it with a `std::string_view` becomes awkward:

```
void use(std::string_view v);

std::string str1;
use(str1); // OK
```

```
my::string str2;  
use(str2); // ERROR
```

Unless `my::string` itself provides an implicit conversion operator towards `std::string_view`, the code above is ill-formed, requiring an explicit conversion instead.

So, what's the problem, just provide the operator, like `std::string` itself does? That's not so simple: the user may not be in control of `my::string` to begin with. For instance, `my::string` may belong to a 3rd party library (Boost, Qt, folly, and so on); *every* sufficiently big codebase has its own string class(es).

This usability impedance is there also if one goes in the opposite direction. Suppose one has a `std::string` object, and wants to use it together with a 3rd party string library. That library also defines its own string view types, and *following the Standard Library's lead*, these string view types feature explicit construction from compatible ranges.

Then:

```
void lib::very_useful_algorithm(lib::string_view v);  
  
std::string str;  
lib::very_useful_algorithm(str); // ERROR
```

There is absolutely no reason why the above shouldn't compile. `std::string` and `lib::string_view` are perfectly compatible, with `lib::string_view` representing the same "platonic concept" of a non-owning `const std::string`. It should be therefore be *implicitly* convertible.

Alas, that's not the case (due to *explicit* construction from a range), and since we cannot modify either datatype (both belong to 3rd parties), we must add explicit conversions *everywhere*. This is completely anti-ergonomic for users.

The idea of adding conversion constructors/operators also does not scale. Again assuming that any 3rd party string view implementation follows [P2499R0]'s design, should the author of `lib::string` add implicit conversion operators towards `lib::string_view`, `std::string_view`, `boost::string_view`, `QStringView`, `llvm::StringRef`, ...? This is poor design (requires $O(N^2)$ conversion constructors/operators), and obviously it's completely impossible to implement in practice (unless one controls all the classes in question).

In Qt, the author [implemented](#) the same semantic change of P2499R0 for Qt's string view classes. The result was that certain implicit conversions got broken, such as from `std::u16string_view` to `QStringView`, despite the two classes representing exactly the same concept (a view over UTF-16/`char16_t` data).



What we are highlighting here is that there is tension between the safety concerns behind the changes proposed by [P2499R0]'s, and the usefulness of string views as a vocabulary interface type -- usefulness that gets dramatically reduced (or removed, one could say) by making the range conversion *unconditionally* explicit.

We could restore most of this usefulness by making the range conversion implicit in the cases where we know it's safe to do. The only problematic aspect is: how to identify those cases?

§ 3. Design Decisions

§ 3.1. How to identify a string(view) class?

Unfortunately there isn't an universal way to identify all and only string(view)-like classes for which we'd like to enable implicit conversions towards string views. As far as their range properties are concerned, a `some_lib::string` and a `std::vector<char>` are identical -- both are contiguous, sized, etc.; a view class like `basic_string_view` cannot distinguish between them based on those range properties alone.

We need therefore a trait, which necessarily needs to be opt-in. The point is that we want to err on the side of caution: for an arbitrary contiguous range we want to keep the conversion explicit. A string(view)-like class can enable the opt-in and become implicitly convertible instead, and string view classes such as `basic_string_view<T>` can be adapted to accept implicit conversions from compatible ranges that have the trait enabled.

There are two alternatives for the design of such a trait.

1. We can introduce a brand new type trait to specialize, and/or a class to inherit from, similar to various other precedents in the Standard Library (for instance, `std::ranges::enable_view<T>`). Any string(view)-like class can then enable the trait by specializing/inheriting.

2. We can use some "unique" feature of string(view)-like types. Specifically, this paper proposes to detect the presence and compatibility of an inner `traits_type` type in order to classify a compatible range as a string(view)-like type. This idea was originally proposed as an option by [\[P2499R0\]](#) but ultimately not chosen; at the time of this writing we do not know the exact reasons.

Both of these options have some downsides.

- In general, both options require an opt-in from existing code. However there is already significant production code that implements approach 2, like `std::string`, `std::string_view`, `folly::fbstring`, `boost::string_view`, `absl::string_view`, etc.; while there is obviously no code that implements 1. These existing datatypes will see their compatibility restored "for free".
- The inner `traits_type` type is not necessarily specific to string(view)-like classes. From simply a naming point of view, it does not have the word "char" or "string" in it or anything like that. The risk here is that some classes might be using it with completely different semantics, and this could result in misdetections.

It sounds pretty unlikely that a *compatible* range declares a *compatible* `traits_type` with a string view, but the range in question is *not* a string(view)-like. The compatibility of `traits_type` is checked in the constraints at `[string.view.cons]/12` (but cf. [\[LWG3857\]](#) the [proposed wording](#)). For reference, in the Standard, `traits_type` is used by `basic_string` and `basic_string_view`; by `basic_regex`; and by the hierarchy of classes in iostreams rooted in `basic_ios`. Of these, only `basic_string` and `basic_string_view` are of a compatible range type and with a compatible trait.

Nonetheless, we sampled various projects in order to back this claim with some real-world data. The results are:

Project	Findings
Folly	Only used in <code>fbstring</code> , which has string semantics.
LLVM (excluding libcxx and tests)	Used by OpenMP in some flag types (<code>kmp_flag</code> , <code>kmp_flag_native</code> , <code>kmp_flag_atomic</code>), which are not ranges.
Abseil	Only used in <code>string_view</code> , which has string view semantics (it's a backport of C++17's <code>std::string_view</code>).
Firefox	According to Searchfox, only 3rd party code (Abseil, protobuf) defines a <code>traits_type</code> type (alias).
Range-v3	Used in the ostream iterators like <code>ostream_iterator</code> .

Project	Findings
Qt	There are no occurrences of <code>traits_type</code> in Qt.
Boost	Boost has a considerable number of usages, about 140 (a complete listing is in Appendix A: traits_type in Boost 1.81), the majority of which are not string related classes at all. This indeed shows that the <code>traits_type</code> name is quite generic, and somehow gets used as as the "default/go-to name" for a type alias for a traits-like parameter of any sort. Still, there's a non-trivial amount of string(view)-like classes. Adding transparent interoperability would be a major usability win. Amongst the string classes, these are compatible with <code>basic_string_view</code>'s range constructor and have a compatible <code>traits_type</code>: ◦ In Container, <code>basic_string</code>. ◦ In Core there's an internal <code>basic_string_view</code> (also used by Url). ◦ In Log, there's an internal <code>basic_string_literal</code>. ◦ In StaticString, <code>basic_static_string</code>. ◦ In Process V2, <code>basic_cstring_ref</code>, <code>key_view</code>, <code>value_view</code> and <code>key_value_pair_view</code>. ◦ In Test, <code>basic_cstring</code>. ◦ In Url, <code>pct_string_view</code>. ◦ In Utility, <code>string_view</code>. ◦ In Wave, <code>flex_string</code>. There's also some non compatible string class: ◦ In Compute, <code>basic_string</code> defines <code>traits_type</code>, but it is not a contiguous range (it's in GPU memory). In the rest of usages, the containing datatype is not a range at all, and/or <code>traits_type</code> is defining some datatype non compatible with <code>basic_string_view</code> anyways.

In conclusion, these results strongly point towards the conclusion that only string(view)-like types are detected: the detection is correct. It is not complete: types such as `QString/QStringView` from Qt or `llvm::StringRef` are string(view)-like types, but are missing a compatible `traits_type` (specifically, they lack `traits_type` entirely).

- A downside of option 1 is the timing aspect. Adding a brand new trait would mean crippling string views for one additional C++ version cycle, until one can use a C++26 (?) compiler that adds the trait and one can use it. Option 2 could (and in our opinion should) be treated as a defect fix instead.

Given the results above, the author's preference would be for option 2, but we would like to gather feedback from SG9, SG16 and LEWG.

§ 3.2. Is this a defect fix?

If option 2 is chosen, we would prefer that the changes proposed by this paper would be considered as a defect fix by implementors and backported to their C++23 modes. For this reason we would not be proposing any new feature-testing macro, nor to bump the value of an existing one. (The change is still entirely detectable by users via type traits, should the need arise.)

Option 1 is fundamentally a brand new API, so it cannot be considered a fix.

§ 4. Impact on the Standard

This proposal is a pure library change. Depending on the option chosen, it may extend/change the proposed resolution to [\[LWG3857\]](#).

The impact is strictly positive: code that was ill-formed (after [\[P2499R0\]](#)) becomes well-formed.

§ 5. Technical Specifications

All the proposed changes are relative to [\[N4928\]](#).

§ 6. Proposed wording

The wording is very similar for both options outlined above. Option 1 would add a new feature-testing macro.

§ 6.1. Option 1: introduce a dedicated type trait

Add to the list in [version.syn]:

```
#define __cpp_lib_is_string_view_like YYYYMMML // also in <string>, <string_view>
```

with the YYYYMMML replaced as usual (year and month of adoption of the present proposal).

Modify [string.view.synop] as follows:

```
// [string.view.template], class template basic_string_view
template<class charT, class traits = char_traits<charT>>
class basic_string_view;
```

```
template<class T>
constexpr bool is_string_view_like = false;
```

```
template<class charT, class traits>
constexpr bool is_string_view_like<basic_string_view<charT, traits>>
```

Add a new section after [string.view.literals]:

23.4 Enablers for implicit string view construction from ranges

[string.view.range.construction]

```
template<class T>
constexpr bool is_string_view_like = false;
```

1. Remarks: Pursuant to [namespace.std], users may specialize is_string_view_like for cv-qualified program-defined types. Such specializations shall be usable in constant expressions ([expr.const]) and have type const bool.

Modify [string.syn] as follows:

```
// [basic.string], basic_string
template<class charT, class traits = char_traits<charT>, class Allocator
    class basic_string;

template<class charT, class traits, class Allocator>
constexpr bool is_string_view_like<basic_string<charT, traits, Allocator>
```

Modify [string.view.template.general] as shown:

```
template<class R>
constexpr explicit(see below) basic_string_view(R&& r);
```

Modify [string.view.cons] as shown:

```
template<class R>
    constexpr explicit(see below) basic_string_view(R&& r);
```

11. Let `d` be an lvalue of type `remove_cvref_t<R>`.

12. *Constraints:*

(12.1) — `remove_cvref_t<R>` is not the same type as `basic_string_view`,

(12.2) — `R` models `ranges::contiguous_range` and `ranges::sized_range`,

(12.3) — `is_same_v<ranges::range_value_t<R>, charT>` is true,

(12.4) — `is_convertible_v<R, const charT*>` is false, and

(12.5) — `d.operator ::std::basic_string_view<charT, traits>()` is not a valid expression, and .

~~(12.6) — if the *qualified-id* `remove_reference_t<R>::traits_type` is valid and denotes a type, `is_same_v<remove_reference_t<R>::traits_type, traits>` is true.~~

13. *Effects:* Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

14. *Throws:* Any exception thrown by `ranges::data(r)` and `ranges::size(r)`.

15. *Remarks:* If the *qualified-id* `remove_reference_t<R>::traits_type` is valid, denotes a type, `is_same_v<remove_reference_t<R>::traits_type, traits>` is true, and `is_string_view_like<remove_reference_t<R>>` is true, then the expression inside `explicit` is equivalent to false. Otherwise, it is equivalent to true.

§ 6.2. Option 2: use a compatible inner `type_traits` type

Modify [string.view.template.general] as shown:

```
template<class R>
    constexpr explicit(see below) basic_string_view(R&& r);
```

Modify [string.view.cons] as shown:

```
template<class R>
    constexpr explicit(see below) basic_string_view(R&& r);
```

11. Let *d* be an lvalue of type `remove_cvref_t<R>`.

12. *Constraints*:

(12.1) — `remove_cvref_t<R>` is not the same type as `basic_string_view`,

(12.2) — *R* models `ranges::contiguous_range` and `ranges::sized_range`,

(12.3) — `is_same_v<ranges::range_value_t<R>, charT>` is true,

(12.4) — `is_convertible_v<R, const charT*>` is false, and

(12.5) — `d.operator ::std::basic_string_view<charT, traits>()` is not a valid expression, and .

~~(12.6) — if the *qualified-id* `remove_reference_t<R>::traits_type` is valid and denotes a type, `is_same_v<remove_reference_t<R>::traits_type, traits>` is true.~~

13. *Effects*: Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

14. *Throws*: Any exception thrown by `ranges::data(r)` and `ranges::size(r)`.

15. *Remarks*: If the *qualified-id* `remove_reference_t<R>::traits_type` is valid, denotes a type, and `is_same_v<remove_reference_t<R>::traits_type, traits>` is true, then the expression inside `explicit` is equivalent to false. Otherwise, it is equivalent to true.

§ Appendix A: `traits_type` in Boost 1.81

```
1 boost/asio/basic_deadline_timer.hpp:148: typedef TimeTraits traits_t
2 boost/asio/basic_waitable_timer.hpp:170: typedef WaitTraits traits_t
3 boost/asio/experimental/basic_channel.hpp:137: typedef typename Trai
4 boost/asio/experimental/basic_concurrent_channel.hpp:137: typedef ty
5 boost/asio/experimental/detail/channel_service.hpp:299: typedef type
6 boost/asio/experimental/detail/channel_service.hpp:396: typedef type
7 boost/asio/experimental/detail/channel_service.hpp:499: traits_typ
8 boost/asio/experimental/detail/impl/channel_service.hpp:196: Sig
9 boost/asio/experimental/detail/impl/channel_service.hpp:224: Sig
```

```
10 boost/asio/experimental/detail/impl/channel_service.hpp:260:      Sig
11 boost/asio/experimental/detail/impl/channel_service.hpp:546:      Sig
12 boost/beast/core/detail/ostream.hpp:122:      using traits_type = typen
13 boost/beast/core/detail/ostream.hpp:51:      using traits_type = typena
14 boost/beast/core/detail/static_ostream.hpp:30:      using traits_type =
15 boost/compute/container/basic_string.hpp:46:      typedef Traits traits
16 boost/container/string.hpp:618:      typedef Traits
17 boost/context/fixedsize_stack.hpp:44:      typedef traitsT traits_type;
18 boost/context/pooled_fixedsize_stack.hpp:127:      typedef traitsT trai
19 boost/context/posix/protected_fixedsize_stack.hpp:46:      typedef trai
20 boost/context/posix/segmented_stack.hpp:46:      typedef traitsT traits
21 boost/context/windows/protected_fixedsize_stack.hpp:38:      typedef tr
22 boost/core/detail/string_view.hpp:350:      typedef std::char_traits<Ch
23 boost/coroutine/posix/protected_stack_allocator.hpp:42:      typedef tr
24 boost/coroutine/posix/segmented_stack_allocator.hpp:43:      typedef tr
25 boost/coroutine/standard_stack_allocator.hpp:41:      typedef traitsT t
26 boost/coroutine/windows/protected_stack_allocator.hpp:35:      typedef
27 boost/date_time/date.hpp:62:      typedef typename calendar::date_trait
28 boost/date_time/date_parsing.hpp:145:      typedef typename std::basi
29 boost/date_time/date_parsing.hpp:314:      typedef typename std::basi
30 boost/date_time/time_duration.hpp:285:      typedef typename base_durat
31 boost/date_time/time_duration.hpp:50:      typedef rep_type traits_type
32 boost/date_time/time_parsing.hpp:57:      typedef typename std::basic_s
33 boost/fiber/future/packaged_task.hpp:58:      >
34 boost/fiber/future/promise.hpp:40:      typedef std::allocator_trai
35 boost/format/alt_sstream.hpp:47:      typedef Tr      traits_typ
36 boost/geometry/srs/projections/impl/pj_ell_set.hpp:179:      typede
37 boost/geometry/srs/projections/spar.hpp:1004:      typedef proj_traits<
38 boost/geometry/srs/projections/spar.hpp:1021:      typedef proj_traits<
39 boost/graph/distributed/adjacency_list.hpp:1308:      traits_type;
40 boost/graph/vertex_and_edge_range.hpp:24:      typedef graph_traits
41 boost/histogram/ostream.hpp:46:      using traits_type = typename OStream
42 boost/interprocess/streams/bufferstream.hpp:274:      typedef typename s
43 boost/interprocess/streams/bufferstream.hpp:345:      typedef typename s
44 boost/interprocess/streams/bufferstream.hpp:417:      typedef typename s
45 boost/interprocess/streams/bufferstream.hpp:72:      typedef CharTraits
46 boost/interprocess/streams/vectorstream.hpp:388:      typedef typename s
47 boost/interprocess/streams/vectorstream.hpp:469:      typedef typename s
48 boost/interprocess/streams/vectorstream.hpp:544:      typedef typename s
49 boost/interprocess/streams/vectorstream.hpp:76:      typedef CharTraits
50 boost/io/ostream_joiner.hpp:48:      typedef Traits traits_type;
51 boost/iostreams/chain.hpp:422:      typedef Tr
52 boost/iostreams/chain.hpp:450:      typedef typename chain_type::traits
```

```

53 boost/iostreams/detail/buffer.hpp:87:     typedef iostreams::char_traits<char> char_traits;
54 boost/iostreams/detail/double_object.hpp:101:     typedef boost::call_traits<char> call_traits;
55 boost/iostreams/detail/double_object.hpp:36:     typedef Metrowerks::char_traits<char> char_traits;
56 boost/iostreams/detail/double_object.hpp:38:     typedef boost::call_traits<char> call_traits;
57 boost/iostreams/detail/double_object.hpp:59:     typedef Metrowerks::char_traits<char> char_traits;
58 boost/iostreams/detail/double_object.hpp:61:     typedef boost::call_traits<char> call_traits;
59 boost/iostreams/detail/double_object.hpp:99:     typedef Metrowerks::char_traits<char> char_traits;
60 boost/iostreams/filter/grep.hpp:50:     typedef char_traits<char_type> char_traits;
61 boost/iostreams/filter/gzip.hpp:463:     typedef char_traits<char> char_traits;
62 boost/iostreams/filter/line.hpp:46:     typedef char_traits<char_type> char_traits;
63 boost/iostreams/filter/stdio.hpp:52:     typedef BOOST_IOSTREAMS_CHAR_TRAITS char_traits;
64 boost/iostreams/filter/symmetric.hpp:76:     typedef BOOST_IOSTREAMS_CHAR_TRAITS char_traits;
65 boost/iostreams/invert.hpp:53:     typedef char_traits<char_type> char_traits;
66 boost/iostreams/read.hpp:120:     typedef BOOST_IOSTREAMS_CHAR_TRAITS char_traits;
67 boost/iostreams/read.hpp:133:     typedef iostreams::char_traits<char_type> char_traits;
68 boost/iostreams/read.hpp:157:     typedef iostreams::char_traits<char_type> char_traits;
69 boost/iostreams/read.hpp:169:     typedef iostreams::char_traits<char_type> char_traits;
70 boost/iostreams/read.hpp:222:     typedef iostreams::char_traits<char_type> char_traits;
71 boost/iostreams/skip.hpp:40:     typedef iostreams::char_traits<char_type> char_traits;
72 boost/iostreams/stream.hpp:31:     typedef Tr char_traits;
73 boost/iostreams/traits.hpp:277:     typedef traits_type char_traits;
74 boost/iostreams/traits.hpp:353:     typedef Tr char_traits;
75 boost/iostreams/write.hpp:81:     typedef BOOST_IOSTREAMS_CHAR_TRAITS char_traits;
76 boost/iostreams/write.hpp:98:     typedef BOOST_IOSTREAMS_CHAR_TRAITS char_traits;
77 boost/lexical_cast/lexical_cast_old.hpp:98:     typedef Traits char_traits;
78 boost/log/detail/attachable_sstream_buf.hpp:61:     typedef typename basic_streambuf::traits_type traits_type;
79 boost/log/expressions/formatter.hpp:59:     typedef typename StreamT::traits_type traits_type;
80 boost/log/expressions/formatters/c_decorator.hpp:134:     typedef Traits traits_type;
81 boost/log/expressions/formatters/c_decorator.hpp:194:     typedef aux::traits_type traits_type;
82 boost/log/expressions/formatters/csv_decorator.hpp:93:     typedef Traits traits_type;
83 boost/log/expressions/formatters/xml_decorator.hpp:92:     typedef Traits traits_type;
84 boost/log/sources/record_ostream.hpp:98:     typedef typename base_traits::traits_type traits_type;
85 boost/log/support/date_time.hpp:70:     typedef typename TimeDurationT::traits_type traits_type;
86 boost/log/utility/formatting_ostream.hpp:127:     typedef TraitsT traits_type;
87 boost/log/utility/string_literal.hpp:58:     typedef TraitsT traits_type;
88 boost/numeric/interval/interval.hpp:45:     typedef Policies traits_type;
89 boost/polygon/polygon_90_set_traits.hpp:56:     typedef typename traits_type::traits_type traits_type;
90 boost/polygon/polygon_90_set_traits.hpp:62:     typedef typename traits_type::traits_type traits_type;
91 boost/process/detail/posix/basic_pipe.hpp:34:     typedef Traits traits_type;
92 boost/process/detail/windows/basic_pipe.hpp:31:     typedef Traits traits_type;
93 boost/process/pipe.hpp:105:     typedef Traits traits_type;
94 boost/process/pipe.hpp:306:     typedef Traits traits_type;
95 boost/process/pipe.hpp:40:     typedef Traits traits_type;

```

```
96 boost/process/pipe.hpp:417:      typedef          Traits          tra
97 boost/process/pipe.hpp:526:      typedef          Traits          tra
98 boost/process/v2/cstring_ref.hpp:55:      using traits_type
99 boost/process/v2/environment.hpp:117:      using traits_type      = key
100 boost/process/v2/environment.hpp:208:      using traits_type      = val
101 boost/process/v2/environment.hpp:303:      using traits_type      = std::
102 boost/process/v2/environment.hpp:487:      using traits_type      = key
103 boost/process/v2/environment.hpp:707:      using traits_type      = val
104 boost/process/v2/environment.hpp:934:      using traits_type      = std
105 boost/range/detail/collection_traits_detail.hpp:288:      t
106 boost/regex/concepts.hpp:301:      typedef typename regex_traits_compute
107 boost/regex/v4/basic_regex.hpp:333:      typedef traits
108 boost/regex/v4/match_results.hpp:384:      typedef ::boost::regex_tra
109 boost/regex/v4/match_results.hpp:396:      typedef ::boost::regex_tra
110 boost/regex/v5/basic_regex.hpp:322:      typedef traits
111 boost/regex/v5/match_results.hpp:355:      typedef ::boost::regex_tra
112 boost/regex/v5/match_results.hpp:367:      typedef ::boost::regex_tra
113 boost/spirit/home/karma/stream/ostream_iterator.hpp:36:      typede
114 boost/static_string/static_string.hpp:833:      using traits_type = Trait
115 boost/test/utis/basic_cstring/basic_cstring.hpp:50:      typedef ut_de
116 boost/test/utis/basic_cstring/basic_cstring.hpp:622:      typedef type
117 boost/test/utis/basic_cstring/compare.hpp:105:      typedef typename b
118 boost/test/utis/nullstream.hpp:46:      typedef typename base_type::tr
119 boost/url/grammar/string_view_base.hpp:86:      typedef std::char_trait
120 boost/utility/string_view.hpp:58:      typedef traits
121 boost/wave/util/flex_string.hpp:1445:      typedef T traits_type;
122 boost/wave/util/flex_string.hpp:2381:      typedef typename flex_string
123 boost/xpressive/detail/core/linker.hpp:117:      typedef typename r
124 boost/xpressive/detail/core/matcher/charset_matcher.hpp:31:      ty
125 boost/xpressive/detail/core/matcher/posix_charset_matcher.hpp:33:
126 boost/xpressive/detail/static/compile.hpp:77:      typedef typename
127 boost/xpressive/detail/static/compile.hpp:90:      typedef typename
128 boost/xpressive/detail/static/transforms/as_set.hpp:107:      typed
129 boost/xpressive/detail/static/visitor.hpp:108:      typedef Traits
130 boost/xpressive/detail/utility/boyer_moore.hpp:42:      typedef Traits
131 boost/xpressive/detail/utility/chset/chset.hpp:34:      typedef Traits
132 boost/xpressive/regex_compiler.hpp:59:      typedef RegexTraits traits_
```

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[LWG3857]

Casey Carter. *basic_string_view should allow explicit conversion when only traits vary*.

Tentatively Ready. URL: <https://wg21.link/lwg3857>

[N4928]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 18 December 2022.

URL: <https://wg21.link/n4928>

[P1989R2]

Corentin Jabot. *Range constructor for std::string_view 2: Constrain Harder*. 17 March 2021.

URL: <https://wg21.link/p1989r2>

[P2499R0]

James Touton. *string_view range constructor should be explicit*. 7 December 2021. URL:

<https://wg21.link/p2499r0>

[QTPATCH]

String view classes: make their range constructors explicit. URL: <https://codereview.qt-project.org/c/qt/qtbase/+448981>